CLOUDGUARD IoT

Unified Autonomous Cloud & IoT Security Platform

API DOCUMENTATION

Proposed API Contract | Base Path: /api/v1

Document scope

Defines the proposed REST API connecting the web dashboard with application services, PostgreSQL, authorized AWS integrations, and the virtual IoT event simulator.

Implementation status

This document describes the proposed system design. It does not imply that every endpoint or integration has already been implemented.

1. Overview

This document defines the proposed API for **CloudGuard IoT — Unified Autonomous Cloud & IoT Security Platform**.

The API connects the web dashboard with application services, PostgreSQL, authorized AWS integrations, and the virtual IoT event simulator.

> ***Note:*** *The endpoints and examples in this document describe the proposed system design. They do not mean that every endpoint or integration has already been implemented.*

2. API Design Principles

- REST-style HTTP endpoints.
- JSON request and response bodies.
- Versioned API structure.
- Server-side validation and authorization.
- Consistent HTTP status codes and error responses.
- Audit logging for security-sensitive operations.
- Response actions restricted to allowlisted test resources.
- Approval required before executing potentially disruptive actions.

3. Base URL

The proposed API base path is:

```
/api/v1
```

Example deployed URL:

```
https://<application-domain>/api/v1
```

4. Authentication and Authorization

Protected endpoints require an authenticated user session or approved access token.

4.1 User Roles

Role	Permissions
Administrator	Manage users, devices, cloud resources, and response workflows
Security Analyst	Review events, investigate incidents, and update permitted incident fields
Viewer	View dashboard, events, resources, and incidents

> *All permissions must be enforced on the server side.*

5. Request and Response Format

Requests containing JSON should use the following headers:

```
Content-Type: application/json
Accept: application/json
```

5.1 Success Response

```
{
  "success": true,
  "message": "Operation completed successfully.",
  "data": {}
}
```

5.2 Error Response

```
{
  "success": false,
  "message": "The request could not be completed.",
  "error": {
    "code": "VALIDATION_ERROR",
    "details": []
  }
}
```

5.3 HTTP Status Codes

Status	Meaning
200	Request completed successfully
201	Resource created successfully
400	Invalid request
401	Authentication required
403	Insufficient permissions
404	Resource not found
409	Conflict
422	Request cannot be processed
500	Internal server error

6. API Endpoints

6.1 Dashboard API

Method	Endpoint	Description
GET	/dashboard/summary	Retrieve security statistics
GET	/dashboard/recent-events	Retrieve recent security events
GET	/dashboard/recent-incidents	Retrieve recent incidents

Example response:

```
{
  "success": true,
  "data": {
    "totalDevices": 0,
    "totalCloudResources": 0,
    "totalSecurityEvents": 0,
    "openIncidents": 0,
    "criticalIncidents": 0
  }
}
```

The values are illustrative placeholders.

6.2 Devices API

Method	Endpoint	Description
GET	/devices	List registered devices
GET	/devices/{deviceId}	Retrieve device details
POST	/devices	Register a new device
PATCH	/devices/{deviceId}	Update device information
DELETE	/devices/{deviceId}	Remove a device registration

Example request:

```
{
  "name": "Temperature Sensor 01",
  "deviceType": "temperature_sensor",
  "sourceType": "simulated",
  "status": "active",
  "location": "Lab A"
}
```

Validation requirements

- Device name is required.
- Device type is required.
- Device identifiers must be unique.
- Only approved source types are accepted.

6.3 Cloud Resources API

Method	Endpoint	Description
GET	/cloud-resources	List monitored cloud resources
GET	/cloud-resources/{resourceId}	Retrieve resource details
POST	/cloud-resources	Register a cloud resource
PATCH	/cloud-resources/{resourceId}	Update resource information
DELETE	/cloud-resources/{resourceId}	Remove a resource registration

Example request:

```
{
  "name": "CloudGuard Test EC2",
  "provider": "AWS",
  "resourceType": "EC2",
  "region": "us-east-1",
  "externalResourceId": "i-example",
  "environment": "test"
}
```

Resource identifiers are illustrative. AWS credentials must never be hardcoded.

6.4 Security Events API

Method	Endpoint	Description
GET	/security-events	List security events
GET	/security-events/{eventId}	Retrieve event details
POST	/security-events	Ingest a security event
POST	/security-events/bulk	Ingest multiple events

Example request:

```
{
  "sourceType": "simulated_iot",
  "sourceId": "sensor-001",
  "eventType": "UNUSUAL_ACTIVITY",
  "severity": "medium",
  "description": "Unusual activity detected by the simulated device.",
  "occurredAt": "2026-10-30T10:30:00Z",
  "metadata": {
    "temperature": 38.5,
    "threshold": 35
  }
}
```

The server validates the source and payload, normalizes the event, assigns an internal identifier, and stores it. The event can then be processed by the risk-assessment and incident-correlation services.

6.5 Risk Assessment API

Method	Endpoint	Description
GET	/risk-assessments	List risk assessments
GET	/risk-assessments/{assessmentId}	Retrieve assessment details
POST	/risk-assessments	Create or request an assessment

Example response:

```
{
  "success": true,
  "data": {
    "id": "assessment-example",
    "score": 72,
    "level": "high",
    "reason": "Illustrative risk factors were detected.",
    "createdAt": "2026-10-30T10:31:00Z"
  }
}
```

The risk score ranges from 0 to 100. The exact scoring formula and severity thresholds must be defined during implementation.

6.6 Incidents API

Method	Endpoint	Description
GET	/incidents	List security incidents
GET	/incidents/{incidentId}	Retrieve incident details
POST	/incidents	Create an incident through the application workflow
PATCH	/incidents/{incidentId}	Update incident information

Example update request:

```
{
  "status": "investigating",
  "notes": "The related events are being reviewed."
}
```

Suggested incident statuses

- open
- investigating
- contained
- resolved
- closed

The server must validate status transitions and record changes in the audit log.

6.7 Response Actions API

Method	Endpoint	Description
GET	/response-actions	List response actions
GET	/response-actions/{actionId}	Retrieve action details
POST	/response-actions	Submit a response-action request
POST	/response-actions/{actionId}/approve	Approve a pending action
POST	/response-actions/{actionId}/reject	Reject a pending action

Example request:

```
{
  "incidentId": "incident-example",
  "targetResourceId": "test-resource-001",
  "actionType": "isolate_test_device",
  "reason": "Controlled containment test."
}
```

Security Requirements

- Only explicitly allowlisted test resources can be targeted.
- Validate target resources and action types on the server.
- Require approval before execution.
- Reject unknown, production, and non-allowlisted targets.
- Record requester, approver, target, timestamp, and execution result.
- Use simulated response actions if real integrations are unavailable.

6.8 Audit Logs API

Method	Endpoint	Description
GET	/audit-logs	List audit records
GET	/audit-logs/{logId}	Retrieve audit record details

Audit logs should record important operations, including:

- Device and resource changes.
- Incident updates.

- Response-action requests.
- Approval and rejection operations.
- Response execution results.

Passwords, tokens, and secret credentials must never be stored in audit logs.

7. Event Processing Workflow

The proposed event-processing workflow is:

1. An authorized AWS integration or IoT simulator submits an event.
2. The API validates the request and source.
3. The application normalizes and stores the event.
4. Risk assessment calculates an explainable risk score.
5. Incident correlation associates the event with an existing incident or creates a new one.
6. The dashboard displays the event, assessment, and incident.
7. An authorized user submits a controlled response request if required.
8. Approval and execution results are recorded in the audit log.

8. Security and Validation

- Validate request bodies and query parameters.
- Enforce authentication and server-side authorization.
- Apply rate limiting to externally accessible ingestion endpoints.
- Use HTTPS in deployed environments.
- Store AWS credentials securely.
- Apply least-privilege IAM permissions.
- Avoid exposing internal stack traces.
- Validate timestamps and payload sizes.
- Prevent duplicate event ingestion where appropriate.
- Never expose secrets in application logs.

9. API Testing

API tests should cover:

- Valid and invalid requests.
- Missing required fields.
- Authentication failures.
- Authorization failures.
- Unknown resource identifiers.
- Duplicate submissions.
- Event ingestion.
- Risk assessment and incident workflows.
- Response-action restrictions.

- Audit-log creation.

10. Implementation Notes

This document represents the proposed API contract. Update endpoint names, fields, authentication details, and response examples to match the actual application during implementation.

Endpoints and integrations that have not been implemented must remain clearly identified as planned or simulated.